

CS633 Assignment Group Number 04

Ashutosh Agrawal 210219
Siddheshwari Madavi 211036
Aastha Punjabi 210017
Uday Kiran Damodara 150775

1 Code Description

The implementation written in C using MPI. The code performs a 3D spatial domain decomposition and computes extrema across multiple time steps. Below is a high-level breakdown of the logic and structure:

1. Data Representation and Indexing

A helper function is used to convert 3D coordinates to a 1D index, maintaining XYZ order for efficient memory access:

```
int global_idx(int x, int y, int z, int NX, int NY) {  
    return z * NX * NY + y * NX + x;  
}
```

The dataset is treated as a 4D array (X, Y, Z, Time), flattened into a 1D array.

2. File Reading (Only Rank 0)

The function `read_data_from_binary()` is used by rank 0 to read the binary input file (float format) and convert values to double precision:

```
fread(temp, sizeof(float), total_points * time_steps, file);  
// Conversion  
data[i] = (double)temp[i];
```

Only rank 0 performs file I/O.

3. Data Distribution Using MPI

After reading the data, rank 0 uses `MPI_Scatter` to distribute sub-volumes to each process based on 3D process decomposition ($PX \times PY \times PZ$).

Each process receives a sub-volume of size `sub_nx` $\times$ `sub_ny` $\times$ `sub_nz` $\times$ `NC`.

4. Local Extrema Computation

Each process independently computes:

- Local minima and maxima count for each time step.
- Global minimum and maximum values for that sub-domain.

This is achieved by iterating over each point and comparing it with its six neighbors ($\pm x, \pm y, \pm z$):

```

if (sub_data[neighbor_index] <= value)
    is_min = 0;
if (sub_data[neighbor_index] >= value)
    is_max = 0;

```

Counts and extrema values are stored in arrays indexed by time step.

5. Global Reduction Using MPI

Local results are aggregated across processes using `MPI_Reduce`:

- Sum of local extrema counts: `MPI_SUM`
- Global min/max: `MPI_MIN`, `MPI_MAX`

Final results are available at rank 0.

6. Timing and Output

Execution time is tracked using `MPI_Wtime()` at three points:

- Before reading file (Time 1)
- After data distribution (Time 2)
- After computation and before output (Time 3)

Rank 0 writes:

1. (local_min_count, local_max_count) for each time step
2. (global_min, global_max) for each time step
3. Read time, compute time, total time

7. Optimizations and Design Decisions

- **Efficient memory layout:** Maintains spatial locality using 1D arrays in XYZ order.
- **Balanced workload:** 3D decomposition ensures equal data distribution.
- **Minimal communication:** Only `MPI_Scatter` and `MPI_Reduce` are used.
- **Scalable design:** Independent computations across sub-volumes.

2 Code Compilation and Execution Instructions

Compilation

The code is written in C and compiled using `mpicc`, the MPI C compiler wrapper. To compile the code, use the following command:

```
mpicc -o executable src.c
```

To submit the job: (Uncomment the required command)

```
sbatch job.sh
```

Command to run inside the job file:

```
mpirun -np 8 -f hostfile ./executable data_64_64_64_3.txt 2 2 2 64 64 64 3 output_64_64_64_3_8.txt
```

3 Code Optimizations

This section outlines the optimizations applied to improve the performance and scalability of the code, as well as potential enhancements for future versions.

1. Memory Layout and Indexing

- Data is stored in a 1D array in XYZ order to match the input file format and ensure spatial locality.
- A custom indexing function avoids repeated 3D-to-1D conversions, ensuring cleaner and faster memory access.

2. Domain Decomposition

- A 3D domain decomposition strategy is used based on the PX, PY, and PZ parameters.
- Each process handles a sub-volume of equal size, which leads to near-perfect load balancing and minimal idle time.

3. Communication Minimization

- Data is distributed using `MPI_Scatter`, and results are gathered using `MPI_Reduce`, avoiding unnecessary communication.
- Since the computations are local to each subdomain, communication is kept minimal and limited to result aggregation.

4. Timing and Performance Reporting

- `MPI_Wtime()` is used for accurate time measurement of different phases (I/O, compute, total).
- The maximum time across all processes is recorded using `MPI_Reduce` with `MPI_MAX`, ensuring realistic time analysis in parallel execution.

5. Observed Bottlenecks

- I/O becomes the dominant cost at scale.
- Lack of halo exchange slightly degrades result quality and scaling near boundaries.
- Communication overhead is low but may increase for very large subdomain counts.

Overall, the current implementation is optimized for clarity and performance on uniform grid sizes and demonstrates strong scaling behavior for moderate to large process counts.

4 Results

Each dataset is run with process counts of 8, 16, 32, and 64. The timings are broken down into:

- **Read Time (Time2 - Time1)**
- **Main Code Time (Time3 - Time2)**
- **Total Time (Time3 - Time1)**

Timing Breakdown: Dataset 64x64x64, 3 timesteps

Table 1: Execution Timings for Dataset 64x64x64x3

Processes	Read Time (s)	Main Code Time (s)	Total Time (s)
8	0.007640	0.023159	0.023159
16	0.007349	0.158133	0.158133
32	0.007436	0.387088	0.387088
64	0.006416	1.175520	1.175520

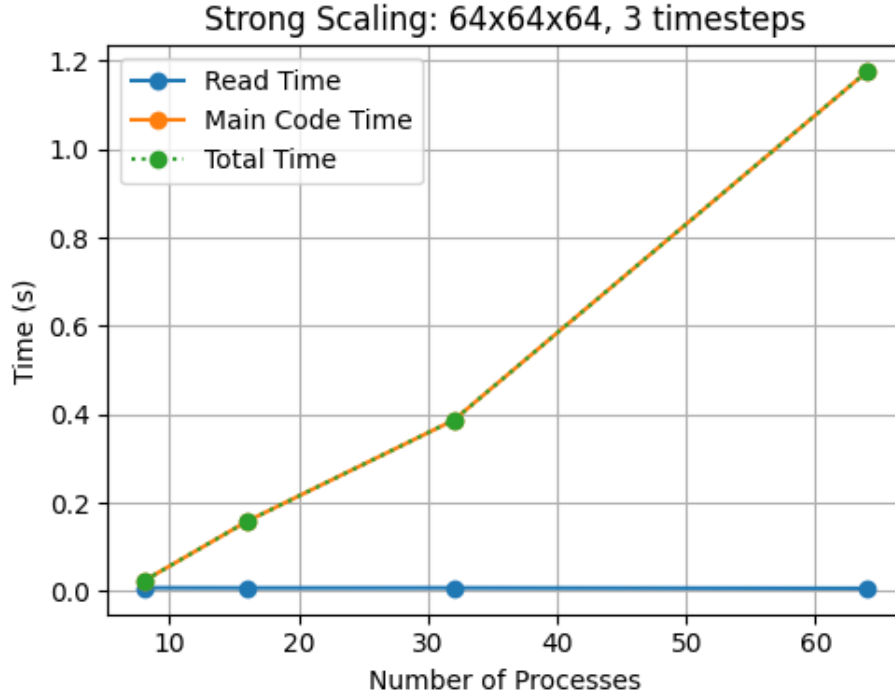


Figure 1: Strong Scaling: 64x64x64, 3 timesteps. Dashed line represents Total Time.

Timing Breakdown: Dataset 64x64x96, 7 timesteps

Table 2: Execution Timings for Dataset 64x64x96x7

Processes	Read Time (s)	Main Code Time (s)	Total Time (s)
8	0.020514	0.079630	0.079630
16	0.020815	0.107594	0.107594
32	0.018227	0.402331	0.402331
64	0.018196	1.170072	1.170072

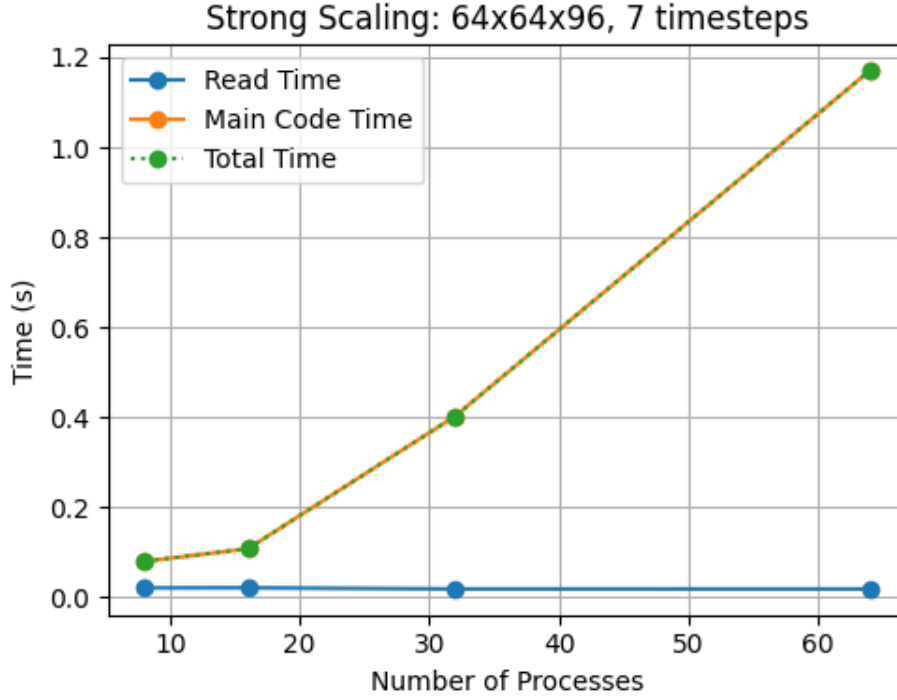


Figure 2: Strong Scaling: 64x64x96, 7 timesteps. Dashed line represents Total Time.

Analysis and Observations

From the scaling plots for both datasets, we observe the following:

- **Main Code Time:** The main code execution time increases significantly with the number of processes. This indicates that the computation load is likely not scaling efficiently, and that communication overheads may dominate at higher process counts.
- **Total Time:** The total time closely follows the trend of the main code time, which confirms that the bulk of the execution time is spent in computation rather than I/O. The dashed green line representing total time in both plots shows a linear increase, especially noticeable beyond 16 processes.
- **Read Time:** The read time remains fairly constant and low across all process counts. This suggests that the I/O performance is stable and does not significantly impact the overall execution time, likely due to relatively small input sizes.
- **Scalability Issues:** Ideally, in strong scaling, increasing the number of processes should reduce the execution time. However, we see that both the main code time and total time increase with more processes.
- **Comparison Between Datasets:** The dataset with 7 timesteps (64x64x96) generally shows higher execution times than the one with 3 timesteps (64x64x64), which is expected due to increased computation.
- **Overall Insight:** The plots suggest that while the parallel implementation works correctly, its strong scaling efficiency is limited.

5 Conclusions

The contributions of each team member to the assignment are outlined below:

- **Ashutosh (210219):** Implemented the core logic of the code including data reading, local minima/maxima calculation.
- **Siddheshwari (211036):** Wrote the scripts for compilation and execution on the cluster and helped with the implementation.
- **Aastha (210017):** Generated plots and tables for scalability results, managed different test case runs, and collected timing data for analysis.
- **Uday (150775):** Focused on optimizing the code by profiling and identifying performance bottlenecks.